Stock.MyPartName [SimTalk]

Returns the current stock of the parts designated by *MyPartName* in the *Store* designated by <Path>.

Remarks

`MyPartName` is the name of the respective part, which you set in the **Configuration Table** of the *Store*.

Type

Read-only attribute

Syntax

```
<Path>.Stock.MyPartName → integer
```

Return Value

The return value has the data type *integer*.

Example

```
print MyStore.Stock.PartRed
// might, for example, return 1
```

See also

Configuration [button]

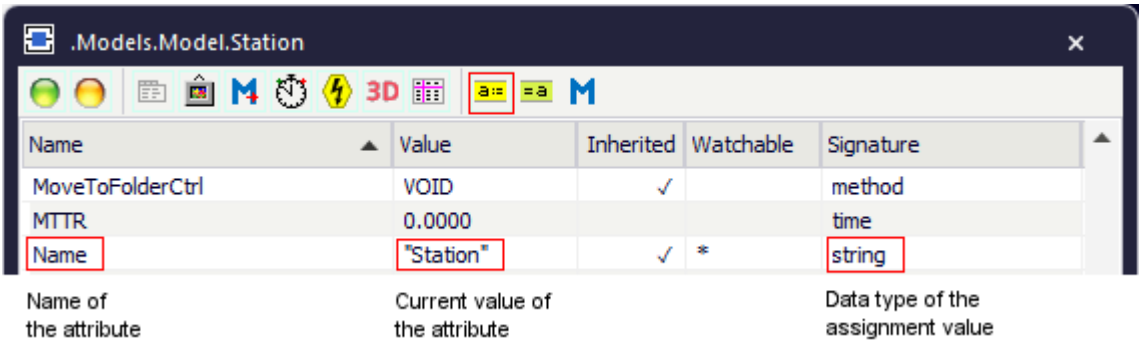
Attributes of the Store

Attributes of the Store

The *Store* provides:

- The *attributes* listed in the table of contents to the left.
- The **Attributes of All Objects**.
- The **Attributes of the Material Flow Objects**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class [general description]**.
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance [general description]**.

You can set the value of an attribute and you can get its value, either with the check boxes, the text boxes and drop-down lists in the dialog windows or by assigning values to the respective attributes.

- To **set the value** of an attribute, you might, for example, type:

```
MyStore.YDim := 10
```

- To **get the value** of an attribute, you might, for example, type:

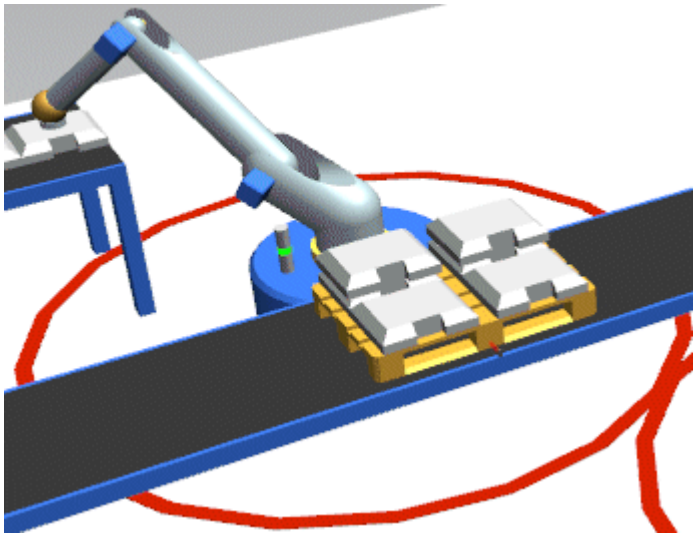
```
print MyStore.YDim
posit := MyStation.Cont.XPos
```

FillWholeLayer [SimTalk] - Store

Sets if the *Store* designated by <Path> always fills an entire layer if its **Z-Dimension** is greater than 1 (`true`) or not (`false`).

Remarks

Plant Simulation always starts a new layer first and only then stacks the parts on the layer below.



Type

Attribute

Syntax

```
<Path>.FillWholeLayer:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Specify `false` to fill each place to its maximum **Z-Dimension** before starting a new layer.

Example

```
MyStore.XDim := 2
MyStore.YDim := 2
MyStore.ZDim := 2
MyStore.FillWholeLayer := true
```

See also

Fill Whole Layer [check box] - Store

Supermarket [SimTalk]

Sets if the *Store* designated by <Path> works as a supermarket (`true`) or not (`false`).

Type

Attribute

Syntax

```
<Path>.Supermarket:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyStore.Supermarket := true
```

See also

Supermarket [check box]

Configuration [button]

Source > **MU selection [drop-down list]** > **Order Controlled [MU selection]**

XDim [SimTalk] - Store

Sets the number of storage places on the x-axis of the *Store* designated by <Path>.

Remarks

The **Capacity** is the product of *XDim* times *YDim* times *ZDim*. The greatest allowed value is ten million.

If you decrease the dimension of the *Store*, make sure that no *MUs* are located on the storage places that will be deleted by this action! Either delete these *MUs* or move them to another storage place on the smaller storage space.

Type

Attribute

Syntax

```
<Path>.XDim:integer
```

Watchable

The attribute is **watchable**.

Assignment Value

You can assign a value of data type *integer*.

Example

```
MyStore.XDim := 10
```

SimTalk

YDim [SimTalk] - Store

ZDim [SimTalk] - Store

See also

X-Dimension [Store]

YDim [SimTalk] - Store

Sets the number of storage places on the y-axis of the *Store* designated by <Path>.

Remarks

The **Capacity** is the product of *XDim* times *YDim* times *ZDim*. The greatest allowed value is ten million.

If you decrease the dimension of the *Store*, make sure that no *MUs* are located on the storage places that will be deleted by this action! Either delete these *MUs* or move them to another storage place on the smaller storage space.

Type

Attribute

Syntax

```
<Path>.YDim:integer
```

Watchable

The attribute is **watchable**.

Assignment Value

You can assign a value of data type *integer*.

Example

```
MyStore.YDim := 10
```

SimTalk

XDim [SimTalk] - Store

ZDim [SimTalk] - Store

See also

Y-Dimension [Store]

ZDim [SimTalk] - Store

Sets the number of storage places on the z-axis of the *Store* designated by <Path>.

Remarks

The *ZDim* permits to stack parts onto other parts.

The **Capacity** is the product of *XDim* times *YDim* times *ZDim*. The greatest allowed value is ten million.

If you decrease the dimension of the *Store*, make sure that no *MUs* are located on the storage places that will be deleted by this action! Either delete these *MUs* or move them to another storage place on the smaller storage space.

Type

Attribute

Syntax

```
<Path>.ZDim:integer
```

Watchable

The attribute is **watchable**.

Assignment Value

You can assign a value of data type *integer*.

Examples

```
MyStore.ZDim := 4
```

```
// Enumerates all the MUs on place (1,1):
var place := Store[1,1]
for var i := 1 to place.NumMU
```

```
   print place.MU(i)
next
```

```
// Returns the topmost MU of the stack:
Store[1,1].Cont
```

```
// Returns the second MU from the top on the place 1,2:
Store[1,2].MU(2)
```

SimTalk

XDim [SimTalk] - Store

YDim [SimTalk] - Store

getStackHeight [SimTalk] - Store

See also

Z-Dimension [Store]

Stack Parts in the Store

Unload Stacked Parts

mu [SimTalk] - PE, Store

PlaceBuffer

PlaceBuffer

Use the object *PlaceBuffer* to process parts on a number of buffer places, which are arranged in a row, one behind the other. It is not part of the built-in objects that the *Toolbox* provides by default.

Image

Description

The *MUs*, which the *PlaceBuffer* processes, have to advance from place to place and can only leave the *PlaceBuffer* after they passed the last place. This way, you can call and access each and every place individually.